AUTOMATA LEARNING LAB

# Python + AI Automation

The Complete Field Guide — O'Reilly Live Training

## Every Automation Is Three Things

Before writing a single line of code, fill in three blanks. If you can say it in one sentence, you have a script.

- **Input?:** A folder, CSV, URL, PDF, clipboard, API response
- **Process?:** Rename, filter, extract, summarize, convert, merge, call an AI model
- **Output?:** A new file, a printed report, an email, a row added to a sheet

> **THE SNIFF TEST — IS IT WORTH AUTOMATING?**
> Ask four questions: (1) Do I do this more than once? (2) Are the steps predictable? (3) Does the input live somewhere a computer can reach? (4) Is the output something a computer can produce? — 4 yeses = strong candidate. 2 or fewer = not worth it yet. "Twice is a coincidence. Three times is a script."

MENTAL MODEL

```
INPUT  ──▶  PROCESS  ──▶  OUTPUT
(data in)   (what to do)    (result)
```

## The Five Categories of Automations

Most beginner-friendly automations fall into one of these buckets.

### 1. Files & Folders

Anything involving your filesystem.

- Rename 300 photos by the date they were taken
- Sort a messy Downloads folder by file type
- Back up a folder to a dated zip file every Friday

### 2. Text & Structured Data

Documents, tables, and PDFs.

- Clean a CSV and drop duplicate rows
- Extract all email addresses from a folder of PDFs
- Convert a .docx report to Markdown

### 3. The Web

Anything that talks to a URL.

- Download the latest stock price once a day
- Scrape a page and save headlines to a file
- Call a public API and turn JSON into a spreadsheet

### 4. App Integrations

Gluing two tools together.

- Read unread Gmail messages and log subjects
- Post a daily summary to Slack
- Sync calendar events into a Notion page

### 5. AI-Powered

Language and judgment tasks.

- Summarize a folder of meeting notes
- Classify incoming support emails by topic
- Turn voice memo transcripts into clean blog drafts

## From Idea to Script in 4 Steps

> THE KEY INSIGHT
>
> The jump students underestimate is NOT writing the code. AI can write the code. It's turning a one-off script into a reusable tool. That's the whole difference between a script and a tool.

### Step 1 — Write the One-Sentence Spec

Fill in: "Take <input>, do <process>, produce <output>." Example: "Take a folder of PDFs, extract every email address, and write them to emails.csv."

### Step 2 — Ask AI to Draft It

PROMPT TEMPLATE (REUSABLE)

```
Write a Python script that:
- Takes <input> as input
- Does <process>
- Produces <output>

Make it runnable from the command line with argparse.
Use only the standard library unless you need <library>.
```

### Step 3 — Run It

BASH

```bash
uv run python scripts/my_tool.py --input ./folder
```

### Step 4 — Turn It into a CLI Tool

- Replace hardcoded paths with argparse arguments
- Save it to scripts/ with a clear name
- Add a one-line --help description

## Running Scripts with uv (PEP 723)

uv is a fast Python package manager that runs single-file scripts with dependencies declared inline — no virtual environment setup needed.

PYTHON — THE INLINE METADATA BLOCK

```python
# /// script
# requires-python = "⩾3.12"
# dependencies = [
#   "openai",
#   "pymupdf",
# ]
# ///
```

### How to Run

BASH

```bash
uv run script.py            # reads metadata, installs deps, runs
uv run script.py arg1 arg2  # pass arguments normally
```

### Claude Projects Prompt for Generating uv Scripts

PROMPT — PASTE INTO CLAUDE PROJECTS INSTRUCTIONS

```
The user will input problems or Python script
descriptions and you will output a single Python
file with the inline metadata to run it standalone
using uv.

Remember to only output the Python file so I can
copy paste and run with: uv run <script>.py
```

### Real Example — PDF Summarizer

PYTHON

```python
# /// script
# requires-python = "⩾3.10"
# dependencies = ["pymupdf", "openai"]
# ///
import sys
from openai import OpenAI
import fitz

def extract_text(filepath):
    doc = fitz.open(filepath)
    return "".join(p.get_text() for p in doc)

def summarize(text):
    client = OpenAI()
    r = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user",
            "content": f"Summarize: {text}"}])
    return r.choices[0].message.content

text = extract_text(sys.argv[1])
print(summarize(text))
```

## The Four R's — Risk Assessment

Before committing to automate something, ask four questions. The answers sort every task into a zone.

- **Reach?:** Can my code get to the data and act on the system?
- **Rules?:** Can I describe the logic in plain steps, or does it need real judgment?
- **Reliability?:** Am I OK with occasional mistakes, or does it have to be perfect?
- **Risk?:** If it misfires, is the worst case recoverable or a disaster?

> **30-SECOND CHECKLIST**
> Before you build: □ I can reach the input without manual steps □ I can describe the rules in a paragraph □ I'm OK with occasional errors, or I have a way to catch them □ If the script misfires, the worst case is easy to undo — 4 checks → build it. 3 checks → build small. 2 or fewer → don't automate yet.

---

### GREEN ZONE — Build It

- Moving data between formats (CSV, JSON, Excel, PDFs, logs)
- Talking to documented APIs (OpenAI, Anthropic, GitHub, Notion, Gmail)
- Scraping public, static web pages
- Running things on a schedule (backups, daily reports)
- LLM language tasks: summarize, classify, extract, rewrite
- Gluing tools together — "when X happens in A, write to B"

---

### YELLOW ZONE — Build Small, Babysit It

- Scraping sites with logins, captchas, or heavy JavaScript
- Automating desktop apps by clicking buttons (breaks on updates)
- Parsing messy/inconsistent documents (scanned PDFs, wild emails)
- Anything that "mostly works" 90% of the time when the other 10% matters
- Long chains of AI calls without verification steps

---

### RED ZONE — Keep Humans in the Loop

- Systems behind strict SSO/2FA without an API

- Tasks that need genuine human judgment every time
- Near-perfect reliability required (medical, legal, safety)
- Irreversible or expensive failure (moving money, mass email, bulk delete)

## The AI Trap

Because this course uses AI heavily, two failure modes matter:

- **Silent Wrongness:** LLMs produce confident, fluent output even when it's wrong. This is the #1 source of broken student automations. Always build in a verification step.
- **Drift:** The same prompt can give different outputs on different days or model versions. Scripts that worked last month may need re-tuning.

> ESSENTIAL HABIT
> Print the AI's output before you act on it — for the first dozen runs at least. Never let an AI script take irreversible action without human review.

## Key Python Libraries

- **AI APIs:** openai, anthropic, ollama
- **Web Scraping:** requests, beautifulsoup4, playwright
- **Data & Viz:** pandas, matplotlib, numpy, plotly
- **PDFs:** pymupdf (fitz), reportlab
- **Office:** python-pptx
- **Email:** imaplib (stdlib), python-dotenv
- **CLI:** argparse (stdlib), sys (stdlib)
- **News/Feeds:** feedparser, newspaper3k

## Quick Reference — 25 Ready-Made Scripts

### File Management

- **file_organizer.py:** Sort files by type into subfolders
- **file_backup.py:** Timestamped folder backups
- **file_image_classifier.py:** AI vision classification of images
- **file_dir_analysis.py:** Directory analysis with charts

### PDF / Documents

- **pdf_summarizer.py:** Summarize any PDF in one command
- **pdf_receipt_extractor.py:** Parse receipts into structured data
- **pdf_invoice_extractor.py:** AI-powered invoice field extraction
- **pdf_paper_summarizer.py:** Download arXiv papers and summarize
- **pdf_chat.py:** Interactive PDF Q&A with AI

### Data / Dashboards

- **stock_dashboard.py:** Interactive stock dashboard (pandas + plotly)
- **api_data_storage.py:** API data → SQLite database
- **csv_dashboard.py:** Interactive dashboard from any CSV
- **etl_pipeline.py:** ETL with validation and transformation
- **log_analyzer.py:** Log analysis with AI + matplotlib charts

### Browser Automation

- **timesheet_automation.py:** Playwright timesheet filling
- **browser_letterboxd.py:** Add movies to watchlist automatically
- **browser_reviews.py:** Scrape and summarize movie reviews
- **browser_imdb.py:** IMDB movie/show data extraction

### Email / Comms

- **news_summarizer.py:** RSS feeds + AI summarization

- **email_urgency_filter.py:** Classify emails by urgency level
- **email_subscriptions.py:** IMAP email management

## The Unified AI Helper Function

A single function that works with OpenAI and Anthropic — copy this into any script or import from ai_tools.py.

PYTHON

```
from openai import OpenAI
import anthropic

def ask_ai(prompt, model="gpt-4o-mini"):
    if "claude" in model:
        client = anthropic.Anthropic()
        r = client.messages.create(
            model=model,
            messages=[{"role":"user",
                       "content":prompt}],
            max_tokens=4000)
        return r.content[0].text
    else:
        client = OpenAI()
        r = client.chat.completions.create(
            model=model,
            messages=[
              {"role":"system",
               "content":"You are a helpful assistant."},
              {"role":"user","content":prompt}])
        return r.choices[0].message.content
```

## Your 5-Minute Action Plan

- Write down 3 repetitive tasks from your week
- For each: fill in Input → Process → Output
- For each: answer the Four R's (reach, rules, reliability, risk)
- Sort them: green / yellow / red
- Build only the green ones this week
- For each red one, write one sentence explaining why it's red

> **REMEMBER**
> The real skill isn't Python. It's instantly recognizing what's automatable and what isn't — and not wasting a weekend on a task that was always in the red zone.